Module 14 – Containerization using Docker

1. Introduction to Docker

What is Docker?

Docker is an open-source containerization platform that enables developers to package applications and their dependencies into lightweight, portable containers. These containers can run consistently across different environments such as development, testing, and production.

Features of Docker

- Lightweight and portable.
- Fast application deployment.
- Platform independent.
- Efficient resource utilization.
- Easy application scaling.
- Isolation of applications.
- Supports microservices architecture.
- Simplifies application deployment.

Benefits of Docker

- Faster software delivery.
- Consistent development and production environments.
- Improved application portability.
- Better resource utilization.
- Easy rollback and updates.
- Simplified dependency management.

2. Docker Architecture

Docker consists of three main components:

Docker Client (CLI)

The Docker Client is the command-line interface used by users to communicate with Docker.

Docker Daemon (dockerd)

The Docker Daemon runs in the background and is responsible for building, running, and managing Docker containers.

Docker REST API

The REST API allows applications to interact with the Docker Daemon programmatically.

3. Docker Commands

Command	Purpose
`docker --version`	Displays the installed Docker version.
`docker info`	Displays detailed Docker system information.
`docker pull <image>`	Downloads an image from Docker Hub.
`docker images`	Lists all downloaded Docker images.
`docker run <image>`	Creates and starts a container from an image.
`docker run -d <image>`	Runs a container in detached (background) mode.
`docker run --name <container-name> <image>`	Runs a container with a custom name.
`docker run -it <image>`	Runs a container interactively.
`docker run -p hostPort:containerPort <image>`	Maps container ports to host ports.
`docker ps`	Lists running containers.
`docker ps -a`	Lists all containers.
`docker stop <container-id>`	Stops a running container.
`docker start <container-id>`	Starts a stopped container.
`docker restart <container-id>`	Restarts a container.
`docker rm <container-id>`	Removes a container.
`docker rmi <image-id>`	Removes a Docker image.
`docker exec -it <container-id> bash`	Opens an interactive shell inside a running container.
`docker logs <container-id>`	Displays container logs.
`docker inspect <container-id>`	Displays detailed information about a container.
`docker build -t image-name .`	Builds a Docker image using a Dockerfile.

4. Docker Run

The **docker run** command creates and starts a new container from a Docker image.

Common Options

- Run a container

```
docker run nginx
```

- Run with a specific name

```
docker run --name mycontainer nginx
```

- Run in detached mode

```
docker run -d nginx
```

- Run interactively

```
docker run -it ubuntu bash
```

- Publish container ports

```
docker run -p 8080:80 nginx
```

- Automatically remove the container after exit

```
docker run --rm ubuntu
```

5. Docker Images

What is a Docker Image?

A Docker Image is a read-only template used to create Docker containers. It contains the application, libraries, dependencies, and configuration files required to run the application.

Image Layers

Docker images are built using multiple read-only layers. Each instruction in a Dockerfile creates a new layer.

Container Layer

A writable layer added when a container is created.

Parent Image

An existing image used as the starting point for creating another image.

Base Image

An image that has no parent image, such as Ubuntu or Alpine.

Docker Manifest

Contains metadata about Docker images such as image version, architecture, and layers.

Container Registry

A storage location for Docker images.

Example:

- Docker Hub
- Azure Container Registry
- Amazon ECR

Container Repository

A collection of related Docker images with different versions.

Creating Docker Images

Interactive Method

Create an image by modifying a running container and committing the changes.

```
docker commit container-id image-name
```

Dockerfile Method

A Dockerfile contains instructions to build Docker images automatically.

Example Dockerfile

```
FROM openjdk:17
COPY app.jar app.jar
ENTRYPOINT ["java","-jar","app.jar"]
```

Build the image

```
docker build -t myapp .
```

Docker Build Context

The build context is the directory containing the Dockerfile and all files required during image creation.

6. Docker Compose

What is Docker Compose?

Docker Compose is a tool used to define and manage multiple Docker containers using a single YAML configuration file.

Benefits

- Manage multiple containers.
- Easy application deployment.
- Simplifies networking.
- Automatic dependency management.

Install Docker Compose

```
docker compose version
```

Create Compose File

Create a file named:

```
docker-compose.yml
```

Example

```
version: "3"

services:
  web:
    image: nginx
    ports:
      - "80:80"
```

Basic Docker Compose Commands

Command	Purpose
`docker compose up`	Starts all services.
`docker compose up -d`	Starts services in detached mode.
`docker compose down`	Stops and removes services.
`docker compose ps`	Lists running services.
`docker compose logs`	Displays service logs.
`docker compose build`	Builds images defined in the Compose file.

7. Docker Engine

Docker Engine is the core runtime responsible for building, running, and managing Docker containers.

It consists of:

- Docker CLI
- Docker Daemon
- Docker REST API

8. Docker Storage

Docker Storage allows containers to store data persistently.

Storage Drivers

Manage how Docker stores image layers and container data.

Data Volumes

Volumes provide persistent storage that remains even after containers are removed.

Create a Volume

```
docker volume create myvolume
```

List Volumes

```
docker volume ls
```

Inspect a Volume

```
docker volume inspect myvolume
```

Remove a Volume

```
docker volume rm myvolume
```

9. Docker Networking

Docker Networking allows communication between containers and external systems.

Default Networks

- Bridge
- Host
- None

List Networks

```
docker network ls
```

Inspect Network

```
docker network inspect bridge
```

Create Network

```
docker network create mynetwork
```

Connect Container

```
docker network connect mynetwork container-name
```

10. Container Orchestration

What is Container Orchestration?

Container Orchestration is the automated management of multiple containers, including deployment, scaling, networking, monitoring, and recovery.

Why Do We Need Container Orchestration?

- Manage large numbers of containers.
- Automatic deployment.
- Automatic scaling.
- High availability.
- Self-healing.
- Load balancing.

Benefits

- Improved scalability.
- Automated deployment.
- Better resource utilization.
- Fault tolerance.
- Simplified management.

Kubernetes Container Orchestration

Kubernetes is an open-source platform used to automate deployment, scaling, networking, and management of containerized applications.

Docker vs Kubernetes

Docker	Kubernetes
Containerization platform	Container orchestration platform
Runs containers	Manages multiple containers
Suitable for small deployments	Suitable for large-scale deployments
Focuses on creating containers	Focuses on managing containers

11. Conclusion

Docker is a powerful containerization platform that enables developers to build, package, and deploy applications consistently across different environments. Features such as Docker Images, Docker Engine, Docker Compose, Storage, Networking, and Container Orchestration simplify modern application development and deployment. Docker, together with Kubernetes, forms the foundation of cloud-native application development and DevOps practices.